

Accuracy and Numerical Stability

We have seen a number of Numerical ODE solvers so far. We have the BDF family, we have the Taylor series family, we have Pade generalizations of Taylor series solvers, and we have two old favorites (Implicit and Explicit Euler) and a few others that are the simplest cases of several families.

The hope is that fancier solvers will let us maintain accuracy with larger steps. We restrict attention to autonomous ODE systems $y' = f(y)$ on $\mathbb{R}^n$ with $f(0) = y_0 \in \mathbb{R}^n$ and assume that f is as smooth as is required for our computations.

Theorems and Definitions.

What happens if I use a non-stiff solver on a stiff problem?

Instability and stability

The rapidly growing oscillations are symptomatic of taking steps that are too large. Not unreasonably this behavior is called instability. It can be predicted and explained by looking at the eigenvalues of the linearization matrix individually.

For a diagonalizable $m \times m$ matrix $A = V \cdot \Lambda \cdot V^{-1}$ the solution of

$$z' = A \cdot z \text{ with } z(0) = z_0$$

is easily written down

$$z(t) = e^{At} \cdot z_0 = V \cdot e^{\Lambda t} \cdot V^{-1} \cdot z_0 = V \cdot \begin{pmatrix} e^{\lambda_1 t} & & \\ & e^{\lambda_2 t} & \\ & & \ddots \\ & & & e^{\lambda_n t} \end{pmatrix} \cdot V^{-1} \cdot z_0.$$

The interpretation of this is that the linear problem decouples in the eigenvector basis.

Explicit Euler, Implicit Euler, and Trapezoid schemes for $z' = A \cdot z$ give

$$\text{Explicit Euler: } z_n = (I + h A)^n \cdot z_0$$

$$\text{Implicit Euler: } z_n = (I - h A)^{-n} \cdot z_0$$

$$\text{Trapezoid: } z_n = \left((I - \frac{h}{2} A)^{-1} (I + \frac{h}{2} A) \right)^n \cdot z_0$$

Substituting $A = V \cdot \Lambda \cdot V^{-1}$ we get the much simpler expressions

$$\text{Explicit Euler: } (I + h A)^n = V \cdot (I + h \Lambda)^n \cdot V^{-1}$$

$$\text{Implicit Euler: } (I - h A)^{-n} = V \cdot (I - h \Lambda)^{-n} \cdot V^{-1}$$

$$\text{Trapezoid: } \left((I - \frac{h}{2} A)^{-1} (I + \frac{h}{2} A) \right)^n = V \cdot \left((I - \frac{h}{2} \Lambda)^{-1} (I + \frac{h}{2} \Lambda) \right)^n \cdot V^{-1}$$

If all the modes decay then the real part of every eigenvalue λ is negative. If the system is stiff then

some of the eigenvalues are well to the left in the left half complex plane. The inner pieces are

$$\begin{aligned}
 \text{Explicit Euler: } & \begin{pmatrix} 1+h\lambda_1 & & & \\ & 1+h\lambda_2 & & \\ & & \ddots & \\ & & & 1+h\lambda_m \end{pmatrix}^n = \begin{pmatrix} (1+h\lambda_1)^n & & & \\ & (1+h\lambda_2)^n & & \\ & & \ddots & \\ & & & (1+h\lambda_m)^n \end{pmatrix} \\
 \text{Implicit Euler: } & \begin{pmatrix} \frac{1}{1-h\lambda_1} & & & \\ & \frac{1}{1-h\lambda_2} & & \\ & & \ddots & \\ & & & \frac{1}{1-h\lambda_m} \end{pmatrix}^n = \begin{pmatrix} \left(\frac{1}{1-h\lambda_1}\right)^n & & & \\ & \left(\frac{1}{1-h\lambda_2}\right)^n & & \\ & & \ddots & \\ & & & \left(\frac{1}{1-h\lambda_m}\right)^n \end{pmatrix} \\
 \text{Trapezoid: } & \begin{pmatrix} \frac{1+h/2\lambda_1}{1-h/2\lambda_1} & & & \\ & \frac{1+h/2\lambda_2}{1-h/2\lambda_2} & & \\ & & \ddots & \\ & & & \frac{1+h/2\lambda_m}{1-h/2\lambda_m} \end{pmatrix}^n = \begin{pmatrix} \left(\frac{1+h/2\lambda_1}{1-h/2\lambda_1}\right)^n & & & \\ & \left(\frac{1+h/2\lambda_2}{1-h/2\lambda_2}\right)^n & & \\ & & \ddots & \\ & & & \left(\frac{1+h/2\lambda_m}{1-h/2\lambda_m}\right)^n \end{pmatrix}
 \end{aligned}$$

Note: Everything is in terms of the scaled variables $z = h\lambda_i$. We have

$$\text{Explicit Euler: } \rightarrow r_{ee}(z) = 1 + z$$

$$\text{Implicit Euler: } \rightarrow r_{ie}(z) = \frac{1}{1-z}$$

$$\text{Trapezoid: } \rightarrow r_t(z) = \frac{1+z/2}{1-z/2}$$

These are the stability functions of these simple single step schemes.

Once we understand that the modes decouple we can define the stability function simply as the result of applying a step of the numerical scheme to the scalar Dahlquist test problem

$$y' = \lambda y$$

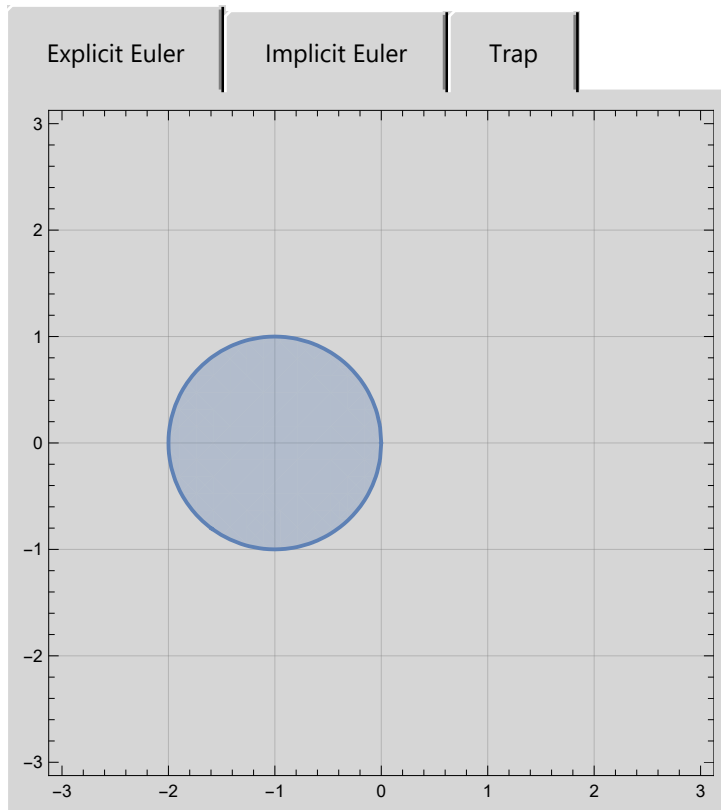
to get $r(\lambda h) y(0)$. If $\text{Re}[\lambda] < 0$ then the analytic solution $e^{\lambda t} y(0)$ decays as $t \rightarrow \infty$. A numerical method that does not reproduce this decay is useless! Applying any one of the three solvers above to the test problem gives $r(\lambda h)^n y(0)$ which decays provided $|r(\lambda h)| < 1$. The stability region of a single step solver is the region in the complex plane where $|r(z)| < 1$.

```

In[ ]:= TabView[{
  "Explicit Euler" →
    RegionPlot[Abs[1 + x + I y] < 1, {x, -3, 3}, {y, -3, 3}, GridLines → Automatic],
  "Implicit Euler" →
    RegionPlot[Abs[ $\frac{1}{1 - (x + I y)}$ ] < 1, {x, -3, 3}, {y, -3, 3}, GridLines → Automatic],
  "Trap" →
    RegionPlot[Abs[ $\frac{1 + 0.5 (x + I y)}{1 - 0.5 (x + I y)}$ ] < 1, {x, -3, 3}, {y, -3, 3}, GridLines → Automatic]
}]

```

Out[]=



An alternative (and frequently a more efficient) way to plot the boundary is to solve $r(z) = e^{i\theta}$ for z . Explicit Euler approximations decay provided $z = \lambda h$ is inside the circle radius 1 centered at the origin. Implicit Euler and Trapezoid approximations decay (as they should) whenever $z = \lambda h$ is in the left half plane where the analytic solution decays.

Analytic solutions decay when the eigenvalues are in the left half plane. Numerical schemes that decay regardless of step size h when the analytic solution decays are A Stable.

Note: Stability has almost nothing to do with accuracy!

Global Accuracy

Approximations for advancing $y' = \lambda y$ from $y(0) = 1$ to $y(1) = e^\lambda$ with n steps of size $h = 1/n$ are

Explicit Euler: $y_{ee} = \left(1 + \frac{\lambda}{n}\right)^n y(0)$

Implicit Euler: $y_{ie} = \left(\frac{1}{1 - \lambda/n}\right)^n y(0)$

Trapezoid: $y_t = \left(\frac{1 + \frac{\lambda}{2n}}{1 - \frac{\lambda}{2n}}\right)^n y(0)$

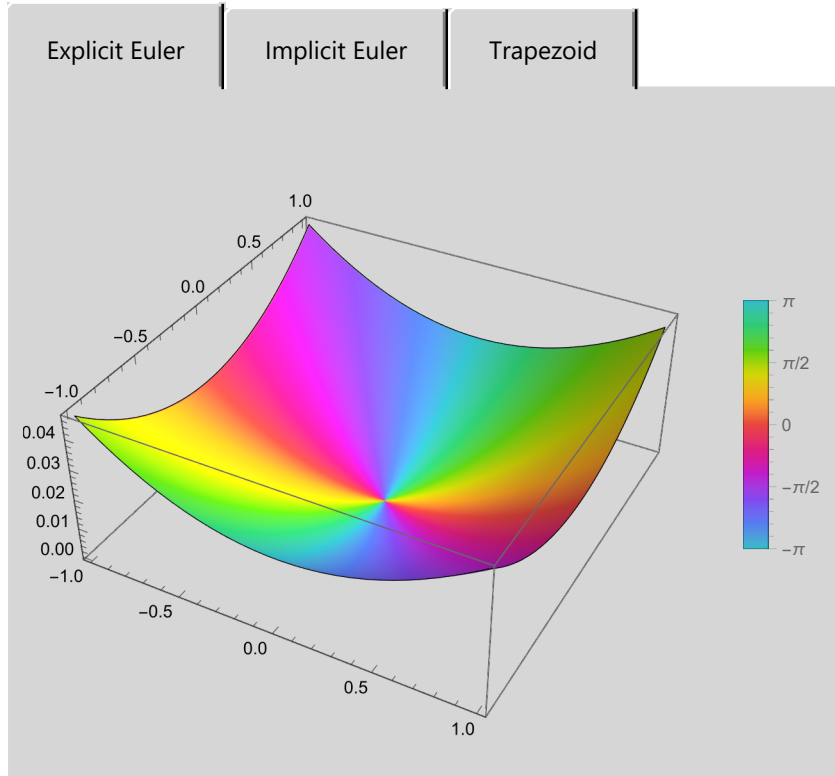
We can see plot the relative global accuracy as a function of $\lambda \in \mathbb{C}$ and $n \in \mathbb{Z}^+$

```

In[ ]:= n = 22;
TabView[{
  "Explicit Euler" → ComplexPlot3D[ $1 - \frac{(1 + \lambda / n)^n}{E^\lambda}$ , {λ, 1},
    PlotLegends → Automatic],
  "Implicit Euler" → ComplexPlot3D[ $1 - \frac{1}{E^\lambda (1 - \lambda / n)^n}$ , {λ, 1},
    PlotLegends → Automatic],
  "Trapezoid" → ComplexPlot3D[ $1 - \frac{(1 + 0.5 \lambda / n)^n}{E^\lambda (1 - 0.5 \lambda / n)^n}$ , {λ, 1},
    PlotLegends → Automatic]
}]

```

Out[]=



Local Truncation Error and Order

The local truncation error (LTE) for a step of length h for $y' = f(t, y)$ with $y(0) = y_0$ is

$$y(h) - y_1$$

where y_1 is the output of the a numerical solver. We are going to do this for our three solvers.

Explicit Euler: $y_1 = y_0 + h f(0, y_0)$

Implicit Euler: $y_1 = y_0 + h f(h, y_1)$

Trapezoid: $y_1 = y_0 + \frac{h}{2} (f(0, y_0) - f(h, y_1))$

As $h \rightarrow 0$ we hope these all become better approximations to $y(h)$. The standard way to evaluate how good the formulas are is to substitute the theoretical solution into the LTE and compute the Taylor Series about $h = 0$.

Explicit Euler LTE: $y(h) - (y_0 + h f(0, y_0))$

Implicit Euler LTE: $y(h) - (y_0 + h f(h, y(h)))$

Trapezoid LTE: $y(h) - (y_0 + \frac{h}{2} (f(0, y_0) - f(h, y(h))))$

We need to remember that $y(0) = y_0$ and $y'(0) = f(0, y_0)$ and $y''(0) = \partial_t f(0, y_0) + \nabla_y f(0, y_0) \cdot y'(0)$ and ...

```
In[ ]:= Series[y[h] - (y[0] + h f[0, y[0]]), {h, 0, 2}]
Series[y[h] - (y[0] + h f[0, y[0]]), {h, 0, 2}] /. {y[0] -> y0, y'[0] -> f[0, y0]}
```

```
Out[ ]:=
(-y0 + y[0]) + (-f[0, y0] + y'[0]) h +  $\frac{1}{2} y''[0] h^2 + O[h]^3$ 
```

```
Out[ ]:=
 $\frac{1}{2} y''[0] h^2 + O[h]^3$ 
```

```
In[ ]:= Series[y[h] - (y[0] + h f[h, y[h]]), {h, 0, 2}]
Simplify[Series[y[h] - (y[0] + h f[h, y[h]]), {h, 0, 2}] /. {
  y[0] -> y0, y'[0] -> f[0, y0],
  y''[0] -> +f(1,0)[0, y0] + f(0,1)[0, y0] y'[0]}]
```

```
Out[ ]:=
(-f[0, y[0]] + y'[0]) h +  $\left( \frac{y''[0]}{2} - y'[0] f^{(0,1)}[0, y[0]] - f^{(1,0)}[0, y[0]] \right) h^2 + O[h]^3$ 
```

```
Out[ ]:=
 $\left( -\frac{1}{2} f[0, y0] f^{(0,1)}[0, y0] - \frac{1}{2} f^{(1,0)}[0, y0] \right) h^2 + O[h]^3$ 
```

```
In[8]:= Series[y[h] - (y0 + h/2 (f[0, y[h]] + f[h, y[h]])), {h, 0, 2}]
Simplify[Series[y[h] - (y0 + h/2 (f[0, y[h]] + f[h, y[h]])), {h, 0, 3}] /. {
  y[0] -> y0, y'[0] -> f[0, y0],
  y''[0] -> f^{(1,0)}[0, y0] + f^{(0,1)}[0, y0] y'[0]}]
```

```
Out[8]=
(-y0 + y[0]) + (-f[0, y[0]] + y'[0]) h +
1/2 (y''[0] - 2 y'[0] f^{(0,1)}[0, y[0]] - f^{(1,0)}[0, y[0]]) h^2 + O[h]^3
```

```
Out[8]=
-1/2 (f[0, y0] f^{(0,1)}[0, y0]) h^2 +
1/12 (2 y^{(3)}[0] - 6 f[0, y0]^2 f^{(0,2)}[0, y0] - 6 f^{(0,1)}[0, y0] f^{(1,0)}[0, y0] -
6 f[0, y0] (f^{(0,1)}[0, y0]^2 + f^{(1,1)}[0, y0]) - 3 f^{(2,0)}[0, y0]) h^3 + O[h]^4
```

In practice ODE solvers control the local “one-step” error and make sure that errors do not amplify.